Synchronization solutions

OS05-E01 Critical-section properties

1. Bounded waiting fails. At most one participant can still be inside, while some contender is always chosen after each exit. Thus exclusion and global progress can hold without a per-requester overtaking bound.
2. No. Others can enter and leave forever. The skipped thread starves without a finite completion bound; the system is not necessarily deadlocked.
3. Use FIFO tickets obtained atomically. Only the queue head enters. With at most n−1 earlier requesters and terminating holders, Pi enters after at most n−1 other admissions.

Liveness and fairness are separate.

OS05-E02 Bounded buffer

1. Wait(empty) before mutex because no capacity is free.
2. After protected removal, signal(mutex), then signal(empty). A woken producer subsequently competes for mutex, preventing simultaneous structure updates.
3. A producer holding mutex waits for consumer-released capacity; the consumer needs that mutex before removing and signaling capacity. The crossed waits deadlock.

Inspect every held-lock blocking path.

OS05-E03 Condition variable

1. CVs do not save signals. Spurious wakeups, broadcasts, or another thread consuming the predicate can invalidate an if-only assumption.
2. Use while(!ready) wait(cv), rechecking under the same mutex after every return.
3. Notifier locks the protecting mutex, changes ready, signals, then unlocks. Waiter checks under the same lock and, if false, atomically unlocks/queues. If notification came first, it sees true and does not wait. If the waiter queued first, the signal finds it.

The CV wakes; the predicate stores state.

Synchronization solutions (continued)

OS05-E04 Peterson correctness and ordering

1. Under atomic seq_cst, assume the current P0/P1 critical sections overlap. If P1's turn=0 is later, P0's flag0=true preceded its own turn write and therefore that later write. P0 does not clear flag before exiting, and neither writes turn inside the current CS. P1 must pass while P0 is still inside, yet flag0=true and turn=0 make its wait condition true, a contradiction. If P0's turn=1 is later, symmetry blocks P0. There is no overlap. P1 writing0 does not make its own turn!=0.
2. Blocked Pi has flag[j]=true and turn=j. Pj keeps turn unchanged inside. After exiting, it clears flag. To enter again, it must write turn=i, giving Pi priority. Pi waits through at most one Pj admission/exit, assuming continued execution.
3. One contender sees the other's false flag and enters. Two contenders have turn equal to one side, permitting the other. Combined with the exclusion and waiting arguments, this establishes all three properties under seq_cst and progressing threads/terminating sections.
4. Ordinary conflicting accesses lack the needed cross-thread synchronization and may be undefined in C/C++. Compiler/CPU reordering or delayed visibility invalidates the sequential interleaving step. Use appropriate atomics, fences, or locks before applying the proof.

The turn persistence, visibility, and waiting-bound arguments must all appear.

OS05-E05 Semaphore states

1. Initially[]. Put A gives empty2/full1/[A]; B gives1/2/[A, B]; C gives0/3/[A, B, C]; get A gives1/2/[B, C]; put D gives0/3/[B, C, D]. Mutex returns 1 after every completed critical section.
2. Distinguish binary free permits0/1 from signed waiter state. T0 wait 1→0 obtains the permit. T1 wait 0→−1 queues. T0 signal−1→0 wakes T1 because incremented $S \leq 0$, and hands it the permit. T1 continues after its original wait without another decrement. S=0 has no free permit, but T1 owns the grant. Its later signal0→1 leaves one free permit only when nobody waits. Signal must increment then wake when $S \leq 0$, not only on0→1.
3. With empty0, producer owns mutex and waits empty. Consumer sees full>0 but waits mutex before removal. These dependencies form the cycle. Wait for capacity outside mutex, then modify slots under mutex.

Synchronization solutions (continued)

The table uses quiescent completed-operation boundaries without reservations, where E+F=3. At general atomic transitions use E+F+P+C=3, with nonnegative free permits E/F and granted producer/consumer reservations P/C, including direct handoff. One wait(empty) changes (3,0,0,0) to (2,0,1,0), so E+F=2 in flight. Signed raw counters also encode queued waiters and are not E/F. Separately require $0 \leq \text{items} \leq 3$ and publication after the corresponding protected update.

OS05-E06 Mesa and lost wakeup

1. Separate unlock and enqueue permit a notifier to signal the still-empty queue between them. Atomic wait removes that interval.
2. Earlier notification sets ready=true under the same lock. The later waiter sees true and skips waiting. An earlier waiter checks and queues atomically, so the notifier finds it.
3. No. Notification does not transfer monitor ownership. Others can change the predicate before its reacquisition. Recheck with while; signal is not a permit.
4. Hoare immediately transfers the monitor and pauses the signaler. The predicate holds at handoff, permitting if under that language's rules, but transfer costs context switching/delayed signaler execution. Mesa keeps the signaler running; the waiter reacquires and rechecks while. POSIX CV and usual Java behavior follow the latter style.

Separate mutex, predicate, condition queue, and signal semantics.

Synchronization solutions (continued)

OS05-E07

States after the six events:

T	m	q	CV	E
T0	—	`[]`	`{C1}`	`{}`
T1	P	`[A]`	`{C1}`	`{}`
T2	P	`[A]`	`{}`	`{C1}`
T3	—	`[]`	`{}`	`{C1}`
T4	C1	`[]`	`{}`	`{}`
T5	—	`[]`	`{C1}`	`{}`

T0 is C1's atomic wait; T1 inserts A; T2 notifies C1; T3 includes P's unlock followed by C2's complete get; T4 reacquires for C1; T5 waits again after finding q empty.

With if, C1 skips predicate rechecking after T4 and attempts an empty pop. The notification reached C1 and it reacquired m. There was neither an extra nor a lost wakeup: a legitimate competitor changed the predicate before reacquisition.

The broken three-step sequence is: (1) C1 sees empty under m, unlocks, and has not registered; (2) P locks, inserts A, signals the empty CV queue, and unlocks; (3) C1 registers and blocks. Now q=[A] while C1 sleeps. This is not legal pthread_cond_wait usage: the API requires ownership on entry and provides an atomic release/wait contract relative to competing acquisition and notification.

Broadcast makes both waiters eligible to reacquire; it does not duplicate A. The first owner passes while and removes A. The second rechecks and waits again. Under the mutex contract, while prevents empty removal, but does not establish FIFO order, a bound on overtaking, or future data arrival. Concrete traces check legal outcomes; the general safety argument is the locked predicate check before every pop, not finite testing alone.

Synchronization solutions (continued)

Verification code appendix: lost_update.py

```
"""Deterministic lost-update interleaving.

Run:
    python lost_update.py
"""


def unsafe_interleaving() -> int:
    balance = 100
    # Two writers both read the old value before either writes back.
    p1_read = balance
    p2_read = balance
    balance = p1_read + 50
    balance = p2_read + 50
    return balance


def serialized_updates() -> int:
    balance = 100
    balance = balance + 50
    balance = balance + 50
    return balance


def main() -> None:
    assert unsafe_interleaving() == 150
    assert serialized_updates() == 200
    print("unsafe=150 serialized=200")


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study